Compte rendu de TP

Commande d’une LED (allumage / extinction) avec la carte
TTGO LoRa32-OLED (ESP32)

Ahmed Hafi
Mohamed Amine Dhaoui
Ayetallah Cherif

1. Objectifs du TP

- Réaliser le montage d'une LED avec sa résistance sur une breadboard et la relier à la carte ESP32.
- Commander la LED par programme : la faire s'allumer puis s'éteindre en boucle (ON / OFF).
- Afficher l'état de la LED sur l'écran OLED intégré à la carte.
- Expliquer le code ligne par ligne.

2. Matériel et logiciels utilisés

Élément	Rôle
Carte TTGO LoRa32-OLED (ESP32)	Microcontrôleur à 2 cœurs avec écran OLED 128×64 intégré ; fournit le signal qui commande la LED
Breadboard (plaque d'essai)	Support de la carte et des composants ; connexions sans soudure
LED rouge	S'allume quand elle est traversée par un courant (sens anode → cathode)
Résistance	En série avec la LED : limite le courant pour ne pas l'abîmer
Fils de connexion (orange, bleu)	Relient la broche GPIO 4 à la résistance et la LED à la masse
Câble USB / micro-USB	Alimente la carte et permet le téléversement du programme
PC + Arduino IDE	Écriture, compilation et téléversement du code
Package esp32 (Espressif)	Ajoute la carte ESP32 à l'IDE
Bibliothèques Adafruit GFX et SSD1306	Affichage du texte sur l'écran OLED (bus I2C, adresse 0x3C)

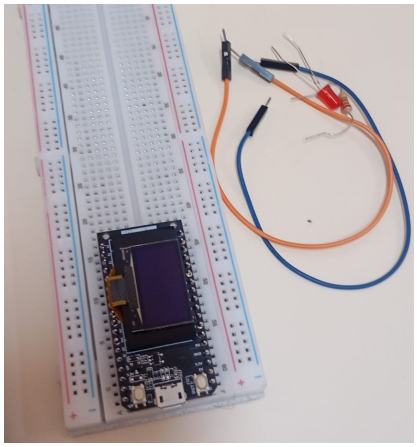


Figure 1 – Matériel : la carte est insérée sur la breadboard ; la LED rouge, la résistance et les fils de connexion sont prêts à être branchés.

3. Montage (câblage)

La carte est insérée sur la breadboard et reliée au PC par le câble USB. La LED est branchée en série avec sa résistance :

- **Broche GPIO 4** de la carte → fil orange → **résistance** → **anode** de la LED (patte longue) ;
- **Cathode** de la LED (patte courte) → fil bleu → **GND** (masse) de la carte ;
- La résistance limite le courant : sans elle, la LED risquerait de griller.

Principe : quand la broche GPIO 4 est à l'état HIGH (3,3 V), un courant traverse la résistance et la LED qui s'allume ; à l'état LOW (0 V), le courant s'arrête et la LED s'éteint.

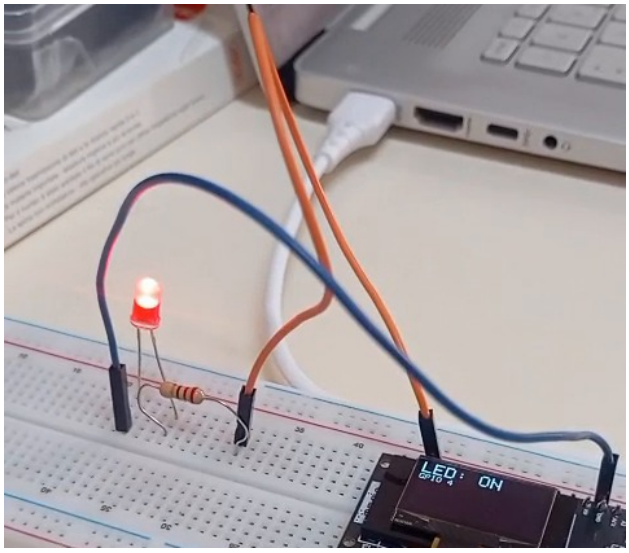


Figure 2 – Montage terminé : LED et résistance reliées à la carte (l'écran indique « LED : OFF, GPIO 4 »).

4. Code source

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define LED_PIN 4
Adafruit_SSD1306 oled(128, 64, &Wire, -1);

void afficher(bool etat) {
  oled.clearDisplay();
   oled.setTextSize(2);
   oled.setCursor(0, 0);
   oled.print("LED: ");
   oled.println(etat ? "ON" : "OFF");
   oled.setTextSize(1);
   oled.println("GPIO 4");
   oled.display();
}

void setup() {
  Wire.begin(21, 22);
  oled.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  oled.setTextColor(SSD1306_WHITE);
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_PIN, HIGH);
  afficher(true);
  delay(1000);
  digitalWrite(LED_PIN, LOW);
  afficher(false);
  delay(1000);
}
```

5. Explication ligne par ligne

Ligne(s)	Explication
1	Inclut **Wire.h** : gestion du bus I2C utilisé pour communiquer avec l'écran.
2	Inclut **Adafruit_GFX.h** : fonctions de base pour écrire du texte.
3	Inclut **Adafruit_SSD1306.h** : pilote de l'écran OLED.
5	Donne le nom `LED_PIN` à la broche **4** (GPIO 4) où est branchée la LED : le code est plus lisible.
6	Crée l'objet `oled` : écran de 128×64 pixels, relié via I2C (`&Wire`), reset = -1 (pas de broche reset).
8	Définit la fonction `afficher`, qui reçoit l'état de la LED (vrai = allumée, faux = éteinte).
9	Efface l'écran avant d'écrire.
10	Taille du texte = 2 (plus grand) pour l'état de la LED.
11	Place le curseur en haut à gauche (x = 0, y = 0).
12	Écrit « LED: » sans retour à la ligne (`print`).
13	Écrit « ON » si l'état est vrai, sinon « OFF », puis passe à la ligne (`etat ? "ON" : "OFF"`).
14	Taille du texte = 1 (petit) pour la ligne suivante.
15	Écrit « GPIO 4 » : la broche utilisée.
16	`oled.display()` envoie le contenu à l'écran : sans cet appel, rien ne s'affiche.
17	Fin de la fonction `afficher`.

Ligne(s)	Explication
19	Début de `setup()`, exécutée une seule fois au démarrage.
20	Initialise l'I2C sur les broches **SDA = 21** et **SCL = 22**, reliées à l'écran.
21	Initialise l'écran à l'adresse **0x3C** ; `SWITCHCAPVCC` : l'écran produit sa tension interne à partir du 3,3 V.
22	Couleur du texte : blanc (pixels allumés).
23	Configure la broche de la LED en **sortie** (`OUTPUT`) : la carte envoie un signal vers la LED.
26	Début de `loop()`, exécutée en boucle infinie.
27	Met GPIO 4 à l'état **HIGH** (3,3 V) : la LED **s'allume**.
28	Affiche « LED: ON » sur l'écran.
29	Attend 1000 ms (1 seconde) : la LED reste allumée.
30	Met GPIO 4 à l'état **LOW** (0 V) : la LED **s'éteint**.
31	Affiche « LED: OFF » sur l'écran.
32	Attend 1 seconde : la LED reste éteinte, puis la boucle recommence.

6. Résultat et conclusion

Après téléversement, la LED s'allume une seconde puis s'éteint une seconde, en boucle. En même temps, l'écran OLED affiche « LED : ON » quand elle est allumée et « LED : OFF » quand elle est éteinte, avec la broche utilisée (GPIO 4). Le comportement de la LED et l'affichage sont donc synchronisés.

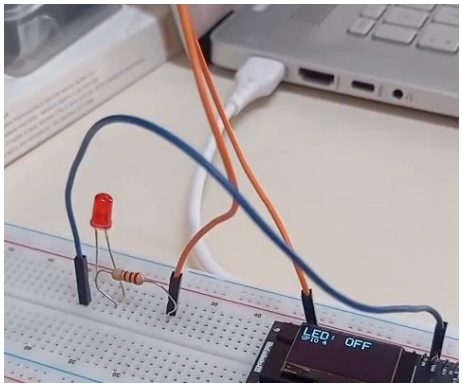


Figure 3 – LED éteinte (LED : OFF)

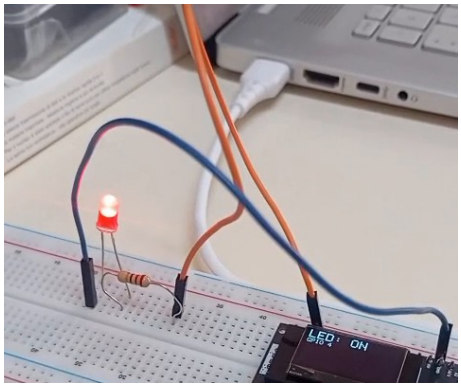


Figure 4 – LED allumée (LED : ON)

Ce TP nous a permis de réaliser un montage simple (LED + résistance sur breadboard), de configurer une broche en sortie, de la commander avec `digitalWrite()` et d'afficher l'état sur l'écran OLED. Les notions de base (entrée/sortie numérique, délai, écran I2C) serviront pour des projets plus avancés.